UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA SEDE NARANJO
FACULTAD DE INGENIERÍA EN SISTEMAS DE INFORMACIÓN
INGENIERÍA EN SISTEMAS DE INFORMACIÓN Y CIENCIAS DE LA COMPUTACIÓN
DESARROLLO WEB
ING. JOSE MIGUEL VILLATORO HIDALGO

MANUAL TECNICO DE TUTORIA CONECTA

Fredy Leonel Morales Albizures 9490-23-16408

Mauricio Isaí Pérez Saban 9490-23-25701

Introducción

Tutoría Conecta es una plataforma que desarrollamos para que un estudiante pueda encontrar un tutor, revisar su oferta y organizar sus clases desde un mismo sitio. El proyecto reúne el catálogo, las reservas, el calendario, el chat y las herramientas de administración. Para nosotros era importante que una solicitud no quedara solamente en pantalla, sino que se guardara con sus fechas, costos y participantes en la base de datos.

En este manual explicamos cómo está organizado el sistema y qué hace cada parte del código. También describimos el uso de TypeScript y SCSS, el modelo de datos y las reglas que evitan que un tutor o un estudiante tenga dos clases al mismo tiempo. La finalidad es que podamos instalar el proyecto, darle mantenimiento y explicar su funcionamiento durante la presentación.

La aplicación utiliza React en el frontend, Express en el backend y PostgreSQL para almacenar la información. El frontend está publicado en Netlify y la API funciona en Render. Los datos se consultan por una API REST, mientras que el chat utiliza Socket.io para enviar y recibir mensajes en tiempo real.

Objetivo y alcance

El objetivo del manual es documentar la implementación que utilizamos en el proyecto final. Se incluyen los tres roles, las integraciones con Google, mapas y correo, la estructura de archivos, las tablas y los controles de acceso. La plataforma calcula presupuestos y organiza servicios educativos; no realiza cobros bancarios ni emite certificaciones académicas.

Organización del trabajo

Trabajamos en pareja. Fredy Leonel Morales Albizures se concentró principalmente en el backend y la integración; Mauricio Isaí Pérez Saban trabajó principalmente en las páginas del

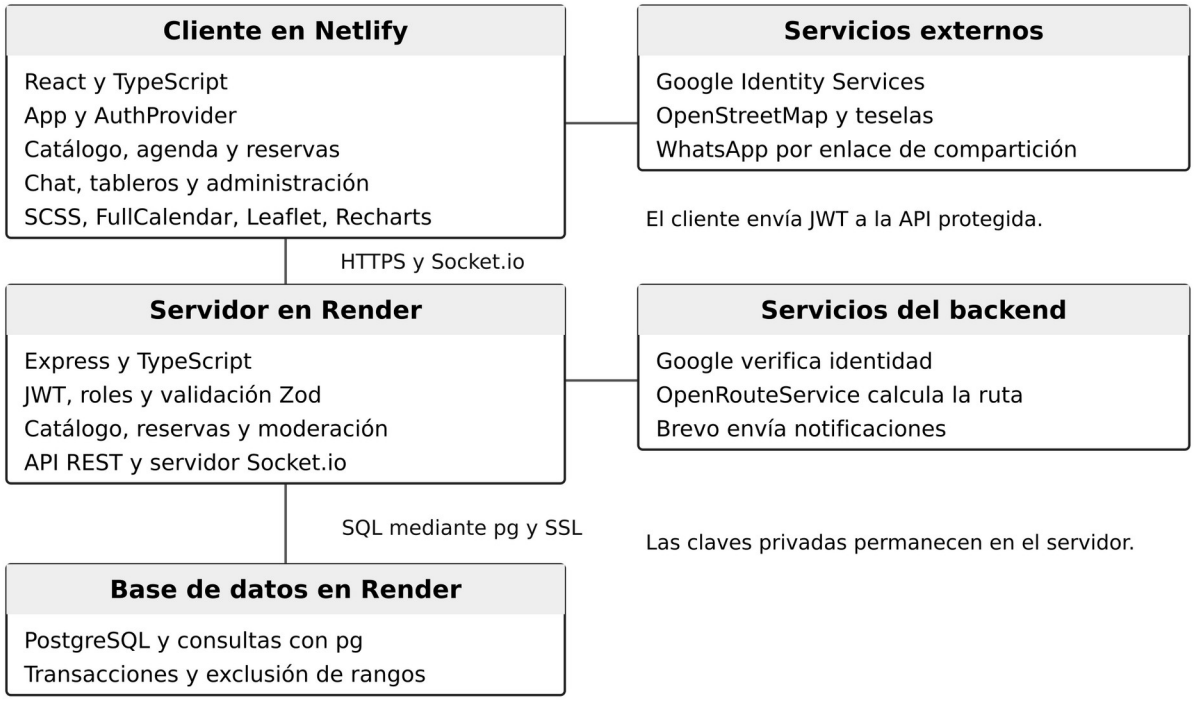
frontend. Las funciones se comprobaron con distintas cuentas y con escenarios de estudiante, tutor y administrador.

Arquitectura y componentes del sistema

Separamos el proyecto en cliente, servidor y base de datos. El navegador presenta los formularios y las pantallas, pero el servidor decide si una operación está permitida. PostgreSQL conserva los registros y aplica restricciones que no dependen de que el usuario tenga abierta la página. La Figura 1 muestra esta distribución y los servicios externos que intervienen.

Figura 1

Esquema conceptual de componentes



Nota. Elaboración propia a partir de server.ts, App.tsx, db.ts, bookings.ts y los módulos de comunicación. Las líneas representan comunicación entre componentes.

El cliente se comunica con la API usando HTTPS y envía el token en las solicitudes protegidas. La API valida la sesión, aplica permisos y ejecuta consultas SQL parametrizadas. El

servidor también solicita rutas a OpenRouteService y notificaciones a Brevo. Google verifica la identidad; la autorización dentro de Tutoría Conecta la controla nuestra aplicación.

Recorrido de una operación

Al confirmar una reserva, BookingForm.tsx envía los datos a /subscriptions. Express verifica al usuario y el módulo bookings.ts revisa la oferta, las fechas y la disponibilidad. Después, una transacción crea el bloque, las sesiones y los intervalos de ambos participantes. La pantalla recibe el resultado y actualiza los horarios. Si una regla falla, el servidor responde con el error y la transacción se revierte.

Distribución de carpetas

Tabla 1

Ubicación de los archivos principales

Ruta	Contenido
backend/src/	Rutas, autenticación, acceso a datos, reglas y servicios.
backend/schema.sql	Definición y ajustes del esquema PostgreSQL.
backend/tests/	Prueba independiente de exclusión y concurrencia.
frontend/src/	Pantallas React, tipos, hooks, API y SCSS.
frontend/public/	Recursos públicos y archivo _redirects.
README.md	Descripción, configuración y comandos generales.

Descripción de los componentes

Componentes del frontend

App.tsx selecciona la vista según la sesión, el estado del perfil y el rol. La navegación entre módulos se controla con estado de React. Las pantallas principales se cargan con lazy y Suspense; no utilizamos React Router en este proyecto.

Tabla 2

Componentes y responsabilidad del frontend

Archivo	Responsabilidad
main.tsx	Monta React y configura GoogleOAuthProvider y AuthProvider.
App.tsx	Inicio de sesión, tema visual y navegación por rol.
auth.tsx	Contexto, reducer, token y actualización de la sesión.
api.ts	Cliente HTTP, encabezados JWT, errores y adjuntos.
Onboarding.tsx	Registro del perfil y ubicación en mapa.
Catalog.tsx	Filtros, ofertas, detalle, reseñas y WhatsApp.
Tutor.tsx	Disponibilidad semanal y publicación de tutorías.
BookingForm.tsx	Configuración del bloque, disponibilidad y presupuesto.
Agenda.tsx	Calendario y acciones de sesiones y suscripciones.
StudentEnrollments.tsx	Consulta administrativa de inscripciones de un estudiante.
Chat.tsx	Conversaciones, mensajes, lectura y archivos.
Classrooms.tsx	Tableros de anuncios, materiales y enlaces.
Admin.tsx	Cuentas, categorías, reportes, apelaciones y gráficos.
Account.tsx	Perfil, políticas, suspensión y denuncias.
Map.tsx	Selección de coordenadas y dibujo de rutas.
types.ts y ui.tsx	Contratos de datos, campos, avisos y hooks compartidos.
styles.scss	Estilos, temas y ajustes responsivos.

Módulos del backend

El backend usa módulos Router de Express. server.ts monta /api/auth para la autenticación y después aplica auth a /api. El catálogo permite consultar categorías durante el registro y aplica active a las demás operaciones. Las rutas de reservas, moderación y chat exigen cuenta activa, y /api/admin agrega la comprobación del rol administrador.

Tabla 3

Módulos y responsabilidad del backend

Archivo	Responsabilidad
server.ts	Servidor HTTP, CORS, Helmet, límites y manejo de errores.
config.ts	Lectura y validación de variables con Zod.
db.ts	Pool PostgreSQL, query y transacciones con tx.
core.ts	JWT, middleware, cifrado, permisos y auditoría.
auth.ts	Google, perfil inicial, /me y apelaciones propias.
catalog.ts	Categorías visibles, ofertas, filtros y disponibilidad.
available-times.ts	Opciones de horario para todas las fechas del bloque.
bookings.ts	Presupuestos, reservas, estados, inasistencias y reseñas.
rules.ts	Recurrencia, ajuste a disponibilidad y distancia geográfica.
chat.ts	Conversaciones, archivos y eventos Socket.io.
classrooms.ts	Acceso y publicaciones de los tableros.
admin.ts	Moderación, categorías, aprobación y métricas.
course-mail.ts y mail.ts	Destinatarios y envío de notificaciones por Brevo.
migrate.ts	Aplica schema.sql a la conexión configurada.
rules.test.ts	Pruebas unitarias de reglas de negocio.
test-mail.ts	Utilidad de prueba de envío de correo.

Uso de TypeScript

Usamos TypeScript en las pantallas y en el servidor porque compartimos datos entre ambas partes. Si cambiamos una propiedad de una reserva, los tipos nos ayudan a encontrar dónde debemos ajustar el código. Esto nos resultó más útil que dejar esas diferencias para descubrirlas al usar la aplicación. Los tipos no reemplazan la validación de lo que recibimos por HTTP; para eso usamos Zod en el backend.

Tipos de datos y componentes

En frontend/src/types.ts definimos User, Offer, Session, Message y Quote, entre otras interfaces. Esto permite reconocer qué propiedades necesita cada pantalla. Los archivos .tsx contienen componentes con JSX y tipos de React. Un ejemplo del proyecto es el siguiente:

```
export interface Block {
  weekday: number;
  start_min: number;
  end_min: number;
}

const [selected, setSelected] = useState<Offer | null>(null);
```

Block expresa el día y los minutos de disponibilidad. En el segundo ejemplo, selected puede contener una oferta o null. Ese estado vacío es necesario cuando todavía no hemos abierto el detalle. No sería correcto tratarlo siempre como una oferta.

Genéricos y respuestas de la API

La función api<T> devuelve Promise<T> y useData<T> reutiliza la carga de información. Por ejemplo, useData<Offer[]> espera una lista de ofertas. El genérico describe el resultado para el compilador, pero no verifica por sí solo que el JSON recibido realmente cumpla esa estructura. En el servidor usamos Zod para comprobar los datos de entrada durante la ejecución.

```
export async function api<T>(
  path: string,
  method = "GET",
  body?: unknown,
): Promise<T>
```

Configuración y validación

Los dos tsconfig.json tienen strict activado. El frontend usa resolución Bundler, JSX react-jsx y noEmit; Vite genera los archivos finales. El backend usa NodeNext y compila src

hacia dist. En core.ts ampliamos Express.Request para incluir req.user. Esto evita repetir una conversión de tipos en cada ruta.

Las entradas externas se validan con esquemas Zod: UUID, fechas, rangos de coordenadas, duración, modalidad y textos. Así se distinguen dos controles: TypeScript revisa el código al compilar y Zod revisa los datos que llegan cuando el sistema está funcionando.

Uso de SASS y SCSS

Los estilos están en frontend/src/styles.scss y se importan desde main.tsx. El paquete sass permite que Vite los transforme en CSS durante la compilación. Aprovechamos SCSS para reutilizar reglas y agrupar los estados de los componentes sin repetir todos los selectores.

Reutilización con mixins

El mixin surface reúne el fondo, el borde, el radio y la sombra de los paneles. Este fragmento corresponde al archivo del proyecto:

```
@mixin surface {
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 22px;
  box-shadow: var(--shadow);
}
```

La regla se aplica con @include surface. También usamos anidamiento y el selector & para estados como &:hover, &:disabled y &.secondary. Si necesitamos cambiar el aspecto común de un panel, podemos modificar el mixin y revisar sus usos.

Temas y diseño responsivo

Los colores se definen como propiedades personalizadas de CSS, por ejemplo --page, --surface y --text. No son variables de Sass con el signo $. Se declaran en :root y cambian en :root[data-theme="dark"]. Esta elección permite alternar el tema mientras la página está abierta, sin volver a compilar.

El archivo contiene reglas @media para anchos de 1000, 700 y 380 píxeles. Estas adaptan columnas, navegación y controles a pantallas pequeñas. Los ajustes del calendario ayudan a mostrar sesiones cortas sin desbordar las tarjetas. También existe una regla prefers-reduced-motion para reducir efectos cuando el dispositivo solicita menos movimiento.

Para mantener los estilos conviene reutilizar las clases existentes y revisar ambos temas. Un cambio debe comprobarse en escritorio y celular, especialmente cuando afecta formularios, ventanas de detalle o eventos del calendario.

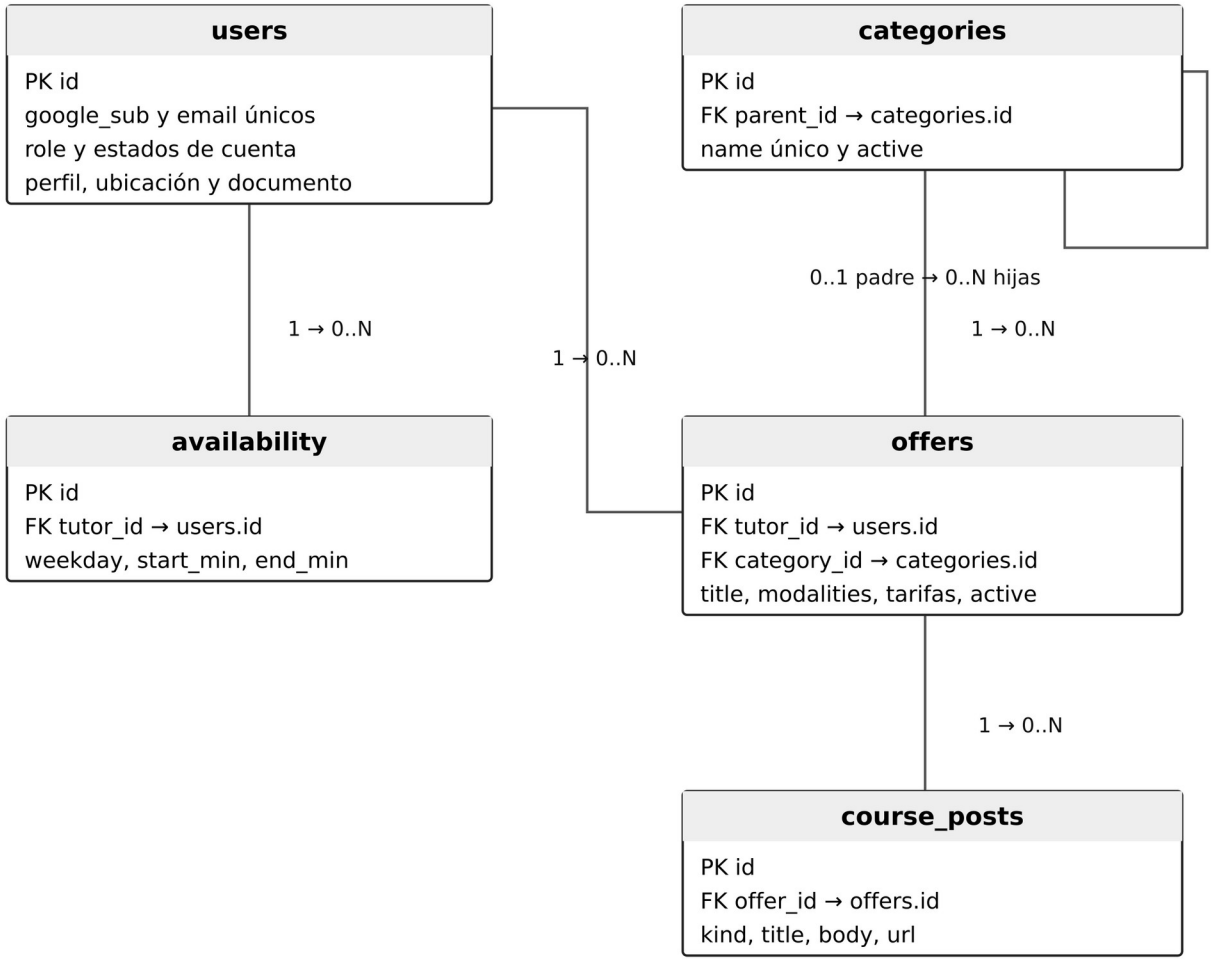
Modelo de datos y diagramas entidad relación

Mauricio y yo organizamos los datos según lo que necesitábamos conservar de una tutoría: quién la ofrece, quién la contrata y qué fechas ocupa. Elegimos PostgreSQL porque necesitábamos relaciones consistentes y una forma de impedir traslapes también al guardar. Trabajamos con consultas SQL mediante pg, sin un ORM. El esquema queda en schema.sql; los diagramas y el diccionario sirven como referencia para localizar cada parte.

Las Figuras 2 a 5 presentan el mismo modelo dividido por función. users, offers y sessions se repiten como referencias entre diagramas; no son tablas duplicadas. PK significa clave primaria, FK significa clave foránea y UNIQUE indica un valor o combinación que no puede repetirse. Las relaciones muestran la cardinalidad desde la tabla referenciada hacia sus registros dependientes.

Figura 2

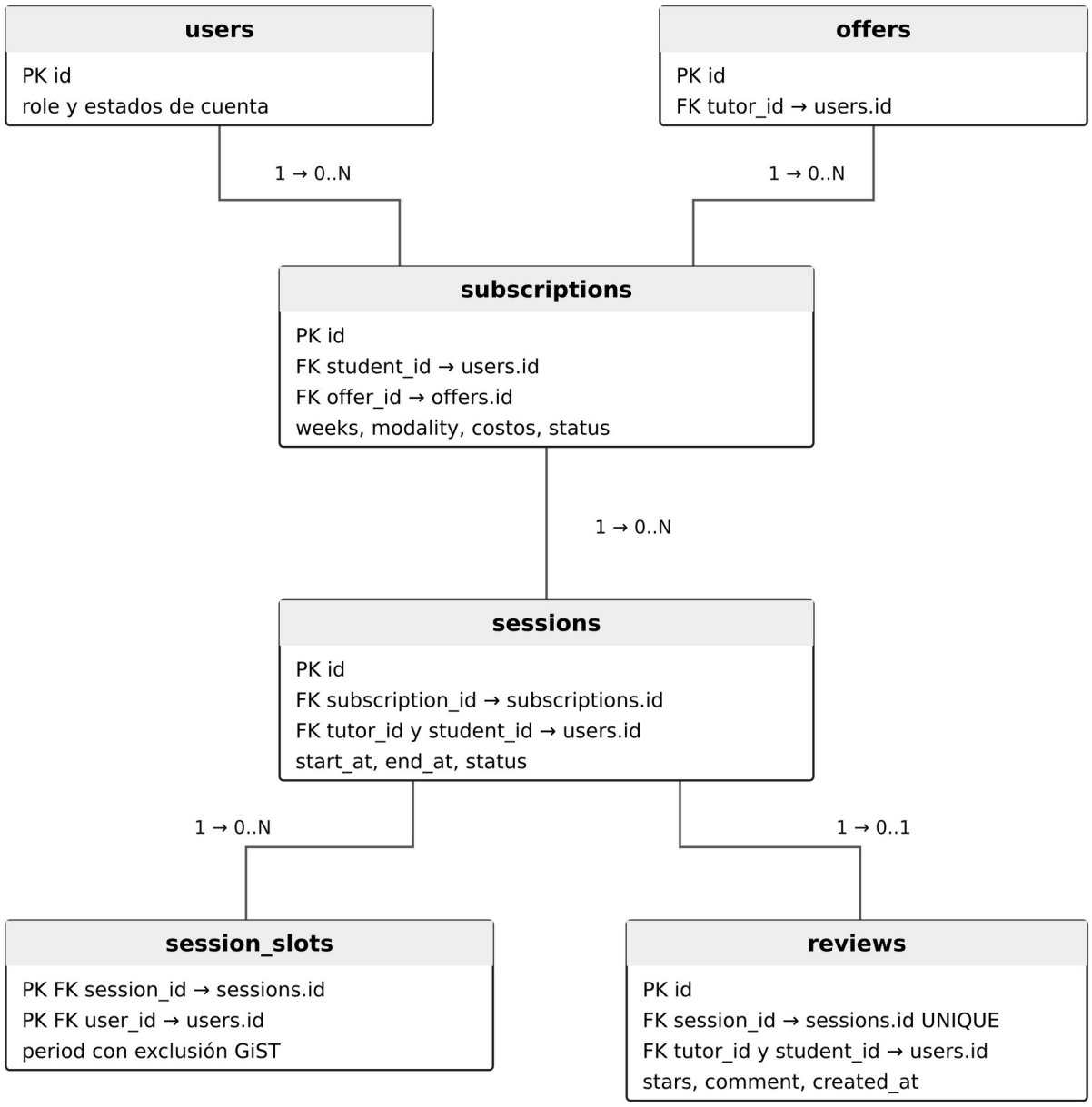
Entidades de usuarios catálogo y publicaciones



Nota. Elaboración propia a partir de schema.sql. Una categoría puede tener un padre opcional. El código administrativo limita la jerarquía a categoría y subcategoría.

Figura 3

Entidades de suscripciones sesiones y reseñas

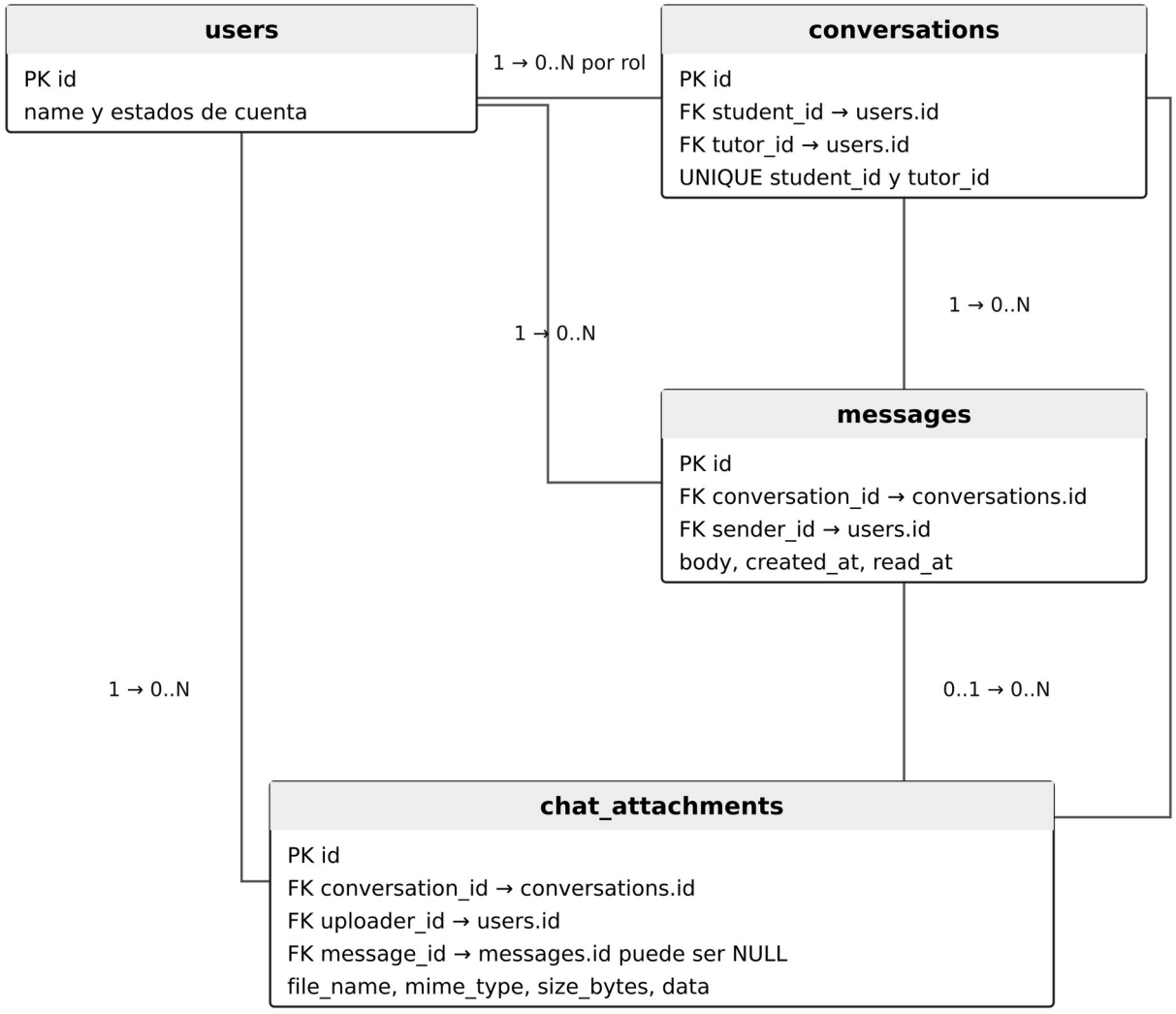


Nota. Elaboración propia a partir de schema.sql. Los roles tutor y estudiante referencian users.

Una sesión puede tener como máximo una reseña. La API crea dos intervalos por sesión.

Figura 4

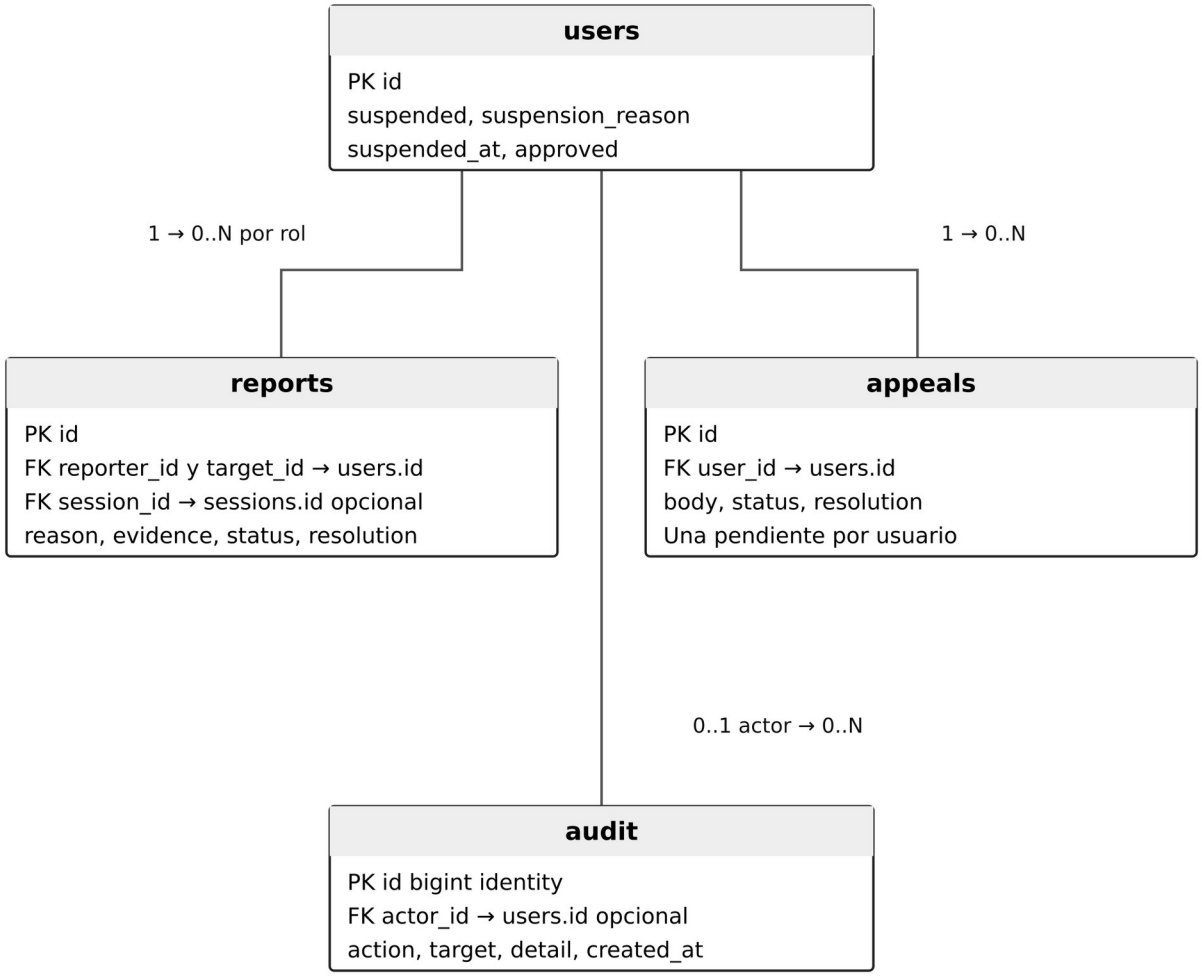
Entidades de conversaciones mensajes y archivos



Nota. Elaboración propia a partir de schema.sql. Cada conversación vincula estudiante y tutor; cada mensaje tiene un remitente. Un adjunto puede estar pendiente de asociarse a un mensaje.

Figura 5

Entidades de reportes apelaciones y auditoría



Nota. Elaboración propia a partir de schema.sql. El reporte puede asociarse a una sesión. actor_id puede ser nulo; target identifica el objetivo como texto y no tiene restricción FK.

Relaciones y restricciones importantes

Separar availability, subscriptions y sessions fue una decisión importante. La disponibilidad indica cuándo suele atender un tutor; la suscripción guarda la contratación y sus costos; las sesiones representan las fechas que realmente reservamos. Si hubiéramos tratado un

bloque de ocho semanas como una sola cita, habría sido más difícil cancelar, registrar una ausencia o revisar un conflicto de una fecha específica.

Lo más delicado fue representar el tiempo ocupado sin revisar solo al tutor. Un estudiante también puede tener una reserva con otra persona. Por eso usamos session_slots y guardamos dos filas por sesión, una por participante. La clave combina session_id y user_id, y la exclusión GiST impide periodos superpuestos para el mismo usuario. Así aplicamos una sola regla a ambos participantes, aunque las reservas correspondan a tutorías diferentes.

También mantuvimos las claves foráneas para no dejar registros sin relación. En la prueba con una categoría y una subcategoría, intentar borrar primero la categoría principal devolvió un conflicto 409. Al borrar primero la subcategoría, pudimos retirar la principal. Cuando existen ofertas relacionadas, la alternativa es desactivar la categoría. Preferimos conservar la información relacionada antes que permitir un borrado que la deje incompleta.

Diccionario resumido de las tablas

Tabla 4

Datos principales y función de cada entidad

Tabla	Datos y función
users	Identidad, rol, perfil, coordenadas, documento cifrado y estados.
categories	Nombre, categoría padre opcional y actividad.
offers	Tutor, categoría, título, descripción, modalidades y tarifas.
availability	Tutor, día semanal y minutos de inicio y fin.
subscriptions	Estudiante, oferta, semanas, modalidad, costos y estado.
sessions	Bloque, participantes, fechas, estado y datos de ausencia.
session_slots	Sesión, participante y rango ocupado.
conversations	Pareja única de estudiante y tutor.
messages	Conversación, remitente, texto, envío y lectura.

Tabla	Datos y función
chat_attachments	Conversación, autor, mensaje opcional, metadatos y bytes.
reviews	Sesión única, participantes, estrellas y comentario.
reports	Denunciante, denunciado, motivo, evidencia y resolución.
appeals	Usuario, argumento, estado y resolución.
audit	Actor, acción, objetivo, detalle JSON y fecha.
course_posts	Oferta, tipo, título, contenido, URL y fechas.

Las fechas de sesiones, mensajes y auditoría usan timestamptz. Los importes se almacenan como numeric para conservar precisión decimal. Las modalidades y afinidades usan jsonb; los adjuntos se guardan como bytea. Las coordenadas son double precision y los intervalos ocupados son tstzrange.

Autenticación autorización y protección de datos

Inicio con Google y emisión del JWT

El botón de Google devuelve credential al frontend. La aplicación lo envía a POST /api/auth/google y el servidor usa verifyIdToken con GOOGLE_CLIENT_ID como audiencia. También exige correo verificado. Verificamos esa identidad en el servidor antes de crear nuestra sesión.

El usuario se crea o se actualiza mediante google_sub. ADMIN_EMAILS determina el rol administrador al crear una cuenta nueva; las demás empiezan como estudiantes y completan el perfil para seleccionar su rol. Incluir un correo después en esa variable no modifica automáticamente el rol de una cuenta que ya existía.

Configuramos el JWT con HS256, el ID del usuario como sujeto, el emisor tutoria-conecta y la audiencia tutoria-web. Dejamos JWT_TTL_SECONDS en 7200 segundos, equivalentes a dos horas. Para probar el vencimiento lo redujimos temporalmente y después

restauramos ese valor. El token identifica la sesión; consultamos el usuario en la base para aplicar su rol y su estado actual.

Validación y permisos

El cliente guarda el token en sessionStorage y lo agrega al encabezado Authorization con el esquema Bearer. El middleware auth exige ese formato y tokenUser valida firma, algoritmo, emisor, audiencia y vencimiento. Después consulta al usuario en la base de datos. El rol y la suspensión se obtienen de ese registro, no de un valor que el navegador pueda cambiar.

active rechaza cuentas suspendidas o sin onboarding. role restringe operaciones según estudiante, tutor o administrador. Además existen comprobaciones de propiedad: un tutor edita sus ofertas, los participantes gestionan sus bloques y member valida el acceso a una conversación. Ocultar un botón es parte de la interfaz; la autorización efectiva está en el backend.

api.ts elimina el token y emite logout cuando una solicitud protegida responde 401. AuthProvider también consulta /auth/me cada 30 segundos. Por eso el cierre por vencimiento se detecta al recibir una respuesta o en la siguiente actualización periódica; no se basa en un contador exacto en pantalla. El 401 del intento de ingreso con Google se presenta como un fallo de ese ingreso.

Datos y límites

El número de identificación se cifra con AES-256-GCM, un IV aleatorio de 12 bytes y la clave DOCUMENT_KEY. safeUser excluye document_cipher y google_sub de las respuestas de usuario. El administrador puede consultar el documento por una ruta específica que registra document.viewed en auditoría. Se debe conservar DOCUMENT_KEY: cambiarla sin una estrategia de migración impediría descifrar los documentos anteriores.

El servidor utiliza Helmet, CORS limitado a CLIENT_ORIGIN y límites de solicitudes. Los valores SQL se pasan como parámetros, y los errores internos se sustituyen por un mensaje general. CORS no reemplaza la autenticación. Tampoco consideramos sessionStorage inmune a XSS; por eso deben mantenerse las validaciones, el tratamiento seguro de contenido y las dependencias revisadas.

Reservas disponibilidad y prevención de traslapes

Generación de sesiones

rules.ts genera fechas a partir de start_date, weeks, days, minute y duration. Recorre un intervalo de semanas multiplicadas por siete días y crea una sesión en cada día seleccionado. Con cuatro semanas y dos días fijos se producen ocho sesiones. El bloque admite 1, 4 u 8 semanas y cada sesión debe durar entre 30 y 480 minutos, sin cruzar al día siguiente.

La fecha inicial representa el comienzo del intervalo. Si se selecciona un día que no coincide con ella, la primera sesión será la primera ocurrencia de ese día dentro del bloque. Las fechas se construyen con UTC−6 y se guardan en formato ISO. ui.tsx y el calendario presentan la hora de Guatemala.

Consulta de disponibilidad

POST /available-times busca los bloques del tutor y los intervalos ocupados tanto por él como por el estudiante. Genera candidatos cada 30 minutos a partir del inicio de cada bloque y exige un horario común para los días seleccionados. Después revisa todas las fechas de la recurrencia. Basta un conflicto para deshabilitar esa opción y mostrar la fecha del problema.

Decidimos que ver un horario disponible no fuera suficiente para darlo por reservado. Entre la consulta y la confirmación, otra persona puede ocuparlo. Por eso usamos /available-

times para orientar la elección y volvemos a validar al guardar en /subscriptions. La protección final queda en la transacción y en la restricción de PostgreSQL.

Transacción de contratación

Antes de guardar validamos la oferta y obtenemos la ruta cuando corresponde. Dentro de tx bloqueamos las filas de los participantes en orden por ID y revisamos de nuevo sus estados, la oferta, la categoría y la disponibilidad. Usamos siempre el mismo orden de bloqueo para reducir conflictos entre solicitudes que llegan al mismo tiempo.

Guardamos la suscripción, sus sesiones y los dos intervalos de cada sesión dentro de la misma transacción. Elegimos ese comportamiento porque no nos servía reservar solo una parte de un bloque: si una fecha tiene conflicto, rechazamos la operación completa. La restricción que protege los intervalos es la siguiente:

```
EXCLUDE USING gist (
  user_id WITH =,
  period WITH &&
)
```

Construimos el periodo con tstzrange(inicio, fin, '[)'). Incluimos el inicio y dejamos fuera el final para permitir clases seguidas: una de 12:00 a 13:00 no debe bloquear otra que empiece exactamente a las 13:00. Ese detalle evita confundir una coincidencia en el límite con un traslape real.

Si una inserción produce el error 23P01, la transacción ejecuta ROLLBACK. No se conserva una parte del bloque ni se deja una suscripción incompleta. El servidor responde 409 con el aviso de traslape para tutor o estudiante. Si todas las inserciones son válidas, COMMIT confirma el bloque completo.

Cambios de estado

Tabla 5

Estados y reglas de las sesiones

Estado	Regla
pending	Se crea al solicitar el bloque y ya ocupa el horario.
approved	El tutor aprueba una suscripción pendiente.
cancelled	La cancelación cambia sesiones pendientes o aprobadas y libera sus intervalos.
completed	El tutor la marca impartida cuando está aprobada y ya terminó.
not_held	Un participante informa una sesión aprobada que ya terminó y registra la causa.

Cancelar un bloque conserva las sesiones impartidas o no impartidas. Cambiar la disponibilidad semanal tampoco cancela reservas existentes. Los intervalos históricos de sesiones terminadas pueden permanecer guardados; al estar en el pasado no impiden reservar una fecha futura.

Mapas distancias y cálculo del presupuesto

El catálogo calcula cercanía mediante kmBetween, que utiliza coordenadas y la fórmula de Haversine. Sirve para filtrar por radio geográfico. Esa distancia es distinta de la ruta vial: no representa calles ni el recorrido real que debe hacer el tutor.

Cuando se selecciona student_home, bookings.ts solicita una ruta driving-car en formato GeoJSON a OpenRouteService. Envía las coordenadas del tutor y del estudiante, obtiene metros y los convierte a kilómetros. Map.tsx transforma el orden longitud-latitud del GeoJSON al orden latitud-longitud de Leaflet y dibuja la polilínea con sus extremos.

Reglas del costo

El costo de clases se obtiene multiplicando la tarifa por hora por la duración en horas y por la cantidad de sesiones. El traslado multiplica la distancia vial por la tarifa por kilómetro y

por la cantidad de sesiones. El total suma ambos resultados. Solo se aplica traslado a domicilio y corresponde al trayecto de ida de cada sesión.

Tabla 6

Ejemplo del presupuesto de un bloque

Concepto	Operación	Resultado
Clases	Q150 × 1 hora × 2 sesiones	Q300.00
Traslados	18.18 km × Q10 × 2 sesiones	Q363.60
Total	Q300.00 + Q363.60	Q663.60

El ejemplo utiliza exactamente 18.18 km. En una ruta real se calcula con la precisión que devuelve el proveedor y al final se formatea el importe. Si la pantalla redondea primero los kilómetros, repetir la operación con ese valor visible puede dar unos centavos de diferencia respecto al total guardado.

La cotización no garantiza el horario ni fija una tarifa anterior. Al confirmar la suscripción, el servidor vuelve a calcular y validar. Si no encuentra una ruta o el proveedor no responde, informa el problema; no utiliza una distancia geográfica como sustituto silencioso de la distancia vial.

Mensajería tableros y notificaciones

Chat privado

La lista y el historial de conversaciones se consultan por REST. Los mensajes nuevos y las confirmaciones de lectura viajan por Socket.io. Cada conexión presenta el JWT mediante handshake.auth.token y se une a una sala user:ID definida por el servidor. No dejamos que el cliente elija la sala de otra persona.

El middleware de conexión se ejecuta al establecerla. Nuestro código agrega validación del token en cada evento y una revisión periódica cada 30 segundos. chat:send comprueba pertenencia, guarda el mensaje y lo emite a las salas de los participantes. chat:read actualiza read_at de mensajes ajenos que todavía no se habían leído.

Adjuntos

Los archivos se suben por HTTP como application/octet-stream. Se admiten PDF, JPEG, PNG y WEBP de hasta 5 MB por archivo. El servidor revisa la firma inicial del archivo, limpia el nombre y guarda los bytes en PostgreSQL. Un mensaje puede incluir hasta cinco identificadores de adjuntos, sin repetidos.

Al enviar el mensaje, la transacción solo enlaza archivos de esa conversación, cargados por el remitente y todavía sin mensaje asociado. La descarga vuelve a comprobar member y responde con Cache-Control private, no-store. El borrado disponible corresponde a archivos temporales propios, no a cualquier archivo ya enviado. Revisar la firma del formato no equivale a un análisis antivirus.

Tableros y correo

course_posts diferencia announcement, material y meeting. Los anuncios pueden no tener enlace; materiales y clases virtuales requieren URL HTTPS. El tutor propietario publica, edita y elimina. Los estudiantes consultan cuando tienen una inscripción aprobada. Las comprobaciones se repiten en la API, aunque alguien intente acceder con una dirección escrita manualmente.

course-mail.ts prepara los destinatarios y mail.ts llama a Brevo. Los enlaces de clase se notifican a estudiantes virtuales aprobados y no suspendidos. También se consultan enlaces existentes al aprobar una inscripción virtual. Al editar una publicación se notifica si pasa a ser meeting o si cambia su URL; un cambio de texto por sí solo no dispara ese envío.

La entrega de correo es una operación externa. Si falla después de guardar una publicación o aprobar la inscripción, la aplicación conserva la operación y muestra una advertencia. La aceptación de Brevo no garantiza que el destinatario haya abierto el mensaje ni que no llegue a correo no deseado.

Moderación reseñas y reportería

Reseñas

Una reseña requiere estudiante propietario de una sesión completed que ya haya terminado. Zod limita stars a un entero entre 1 y 5 y exige comentario. La restricción UNIQUE de session_id evita una segunda valoración de la misma sesión. La operación registra review.created en auditoría.

El promedio y el número de reseñas se calculan al consultar las ofertas. No se mantiene un promedio manual separado. La reportería también consulta las reseñas para los rankings. Las pantallas muestran el valor actualizado cuando recargan sus datos; el catálogo no recibe un evento Socket.io específico de actualización de calificaciones.

Denuncias suspensión y apelación

El formulario de reportes ofrece contactos asociados a sesiones o conversaciones y permite evidencia textual. La API valida el destinatario como UUID, el motivo y el contenido, e impide denunciarse a uno mismo. En el estado not_held puede generarse una denuncia de inasistencia contra la otra parte cuando corresponde.

El administrador registra el motivo de suspensión y la fecha. El usuario puede seguir iniciando sesión, pero la vista cambia a Cuenta suspendida y el backend bloquea las operaciones activas. Solo quedan disponibles las rutas permitidas del flujo de cuenta, incluido consultar y presentar apelaciones. La suspensión también provoca la desconexión del chat.

La apelación tiene estado pending, accepted o rejected. Al aceptar, la transacción actualiza el caso, desactiva suspended y registra la acción. Rechazar no reactiva. Se conserva la información del bloqueo como historial y la bandera suspended determina el acceso actual.

Indicadores administrativos

GET /admin/analytics recibe month con formato AAAA-MM. Las horas por categoría suman sesiones approved y completed cuya fecha de inicio pertenece al mes. El gráfico de estados cuenta sesiones del mismo periodo. Las canceladas y no impartidas no se agregan a las horas contratadas que muestra ese indicador.

El ranking es histórico y no depende del mes elegido. Incluye tutores que tienen reseñas y presenta promedio y cantidad. Si solo hay un tutor valorado, puede aparecer tanto en mayor como en menor calificación. Las consultas permiten abrir sus comentarios desde el panel.

Referencia de la API y eventos

La base de producción es https://tutoria-conecta.onrender.com/api. Las rutas siguientes se escriben relativas a esa base. Salvo el ingreso con Google, las solicitudes HTTP requieren el JWT de la aplicación. La entrada de Google lleva el token de identidad en credential y en Authorization. Los cuerpos usuales son JSON; los adjuntos usan bytes.

Tabla 7

Rutas de autenticación catálogo y disponibilidad

Método y ruta	Acceso y propósito
POST /auth/google	Token de Google. Verificar identidad y emitir sesión.
GET /auth/me	Usuario autenticado. Consultar perfil propio.
POST /auth/onboarding	Cuenta sin perfil completo. Guardar datos iniciales.
GET y POST /auth/appeals	Consultar apelaciones propias o presentar una estando suspendido.
GET /categories	Cuenta autenticada no suspendida. Categorías activas.

Método y ruta	Acceso y propósito
GET /offers	Cuenta activa. Ofertas y filtros.
GET /offers/mine	Tutor activo. Ofertas propias.
POST /offers	Tutor aprobado. Crear oferta.
PUT /offers/:id	Tutor aprobado y propietario. Editar oferta.
GET /availability/:id	Cuenta activa. Disponibilidad semanal.
PUT /availability	Tutor activo. Reemplazar disponibilidad propia.
GET /tutors/:id/reviews	Cuenta activa. Consultar valoraciones.

Tabla 8

Rutas de reservas y comunicación

Método y ruta	Acceso y propósito
POST /available-times	Estudiante activo. Opciones del bloque.
POST /quote	Estudiante activo. Calcular presupuesto.
POST /subscriptions	Estudiante activo. Crear reserva completa.
PATCH /subscriptions/:id	Participantes. Cancelar; solo tutor aprueba.
GET /sessions	Cuenta activa. Sesiones propias.
POST /sessions/:id/complete	Tutor propietario. Marcar impartida.
POST /sessions/:id/not-held	Participante. Registrar sesión no impartida.
POST /sessions/:id/reviews	Estudiante propietario. Valorar sesión impartida.
GET y POST /conversations	Listar propias; estudiante inicia con tutor disponible.
GET /conversations/:id/messages	Participante. Historial de mensajes.
POST /conversations/:id/attachments	Participante. Subir archivo.
GET y DELETE /conversations/:id/attachments/:attachmentId	Descarga para participante; borrado de temporal propio.
GET /classrooms	Tutor o estudiante activo. Tableros disponibles.
GET y POST /classrooms/:id/posts	Lectura autorizada; creación por tutor propietario.
PUT y DELETE /classrooms/:id/posts/:postId	Tutor propietario. Modificar publicaciones.

Tabla 9

Rutas de moderación y administración

Método y ruta	Acceso y propósito
GET /contacts	Cuenta activa. Contactos para el formulario.
POST /reports y GET /reports/mine	Cuenta activa. Crear y consultar denuncias propias.
GET /admin/users	Administrador. Cuentas y perfiles.
GET /admin/users/:id/document	Administrador. Descifrar documento con auditoría.
GET /admin/users/:id/subscriptions	Administrador. Bloques de un estudiante.
PATCH /admin/users/:id/approve	Administrador. Aprobar tutor completo.
POST /admin/users/:id/suspend	Administrador. Suspender cuenta no administradora.
GET y POST /admin/categories	Administrador. Listar o crear categorías.
PUT y DELETE /admin/categories/:id	Administrador. Editar o eliminar categoría.
GET /admin/reports	Administrador. Denuncias y filtro por usuario.
PATCH /admin/reports/:id	Administrador. Resolver denuncia abierta.
GET /admin/appeals	Administrador. Bandeja de apelaciones.
PATCH /admin/appeals/:id	Administrador. Aceptar o rechazar pendiente.
GET /admin/audit	Administrador. Últimos 300 registros.
GET /admin/analytics?month=AAAA-MM	Administrador. Rankings y métricas mensuales.

Eventos y respuestas

Socket.io usa chat:send para enviar, chat:message para recibir, chat:read para actualizar lecturas y account:changed para solicitar revisión de cuenta. Las confirmaciones de eventos utilizan ok y, si hay error, message. El JWT de la conexión se envía en auth del handshake, no mediante el cuerpo JSON de la API REST.

Las respuestas HTTP usan 201 al crear y 204 cuando no hay contenido. Los errores principales son 400 por datos inválidos, 401 por sesión inválida, 403 por falta de permiso y 404

por recurso o ruta no disponible. Los conflictos de reserva, duplicados y relaciones de base de datos se presentan como 409. Un fallo del proveedor de rutas puede responder 503.

Instalación configuración y despliegue

Requisitos y variables

Se necesita Node.js 22.12 o superior, npm y una conexión PostgreSQL que permita btree_gist. El backend valida sus variables al iniciar. En db.ts la conexión aplica sslmode=verify-full, por lo que la instancia de PostgreSQL debe ofrecer SSL con certificado válido. No debe deshabilitarse la verificación para ocultar un error de conexión.

Tabla 10

Variables de configuración

Variable	Uso
DATABASE_URL	Conexión privada a PostgreSQL.
JWT_SECRET	Secreto de al menos 32 caracteres.
JWT_TTL_SECONDS	Duración entre 60 y 86400 segundos; habitual 7200.
DOCUMENT_KEY	Clave hexadecimal de 64 caracteres para AES-256.
GOOGLE_CLIENT_ID	Audiencia para verificar Google.
CLIENT_ORIGIN	Origen autorizado del frontend.
ADMIN_EMAILS	Correos administradores separados por comas.
ORS_API_KEY	Clave del proveedor de rutas.
BREVO_API_KEY	Clave del servicio de correo.
MAIL_FROM_EMAIL y MAIL_FROM_NAME	Remitente autorizado y nombre visible.
PORT	Puerto del backend; valor local predeterminado 4000.
VITE_API_URL	URL base de la API en el frontend.
VITE_GOOGLE_CLIENT_ID	Identificador público de Google para el frontend.

backend/.env contiene las variables privadas de desarrollo. frontend/.env solo debe contener configuración pública. Vite incorpora las variables VITE_ al paquete del navegador; nunca deben ponerse allí JWT_SECRET, DOCUMENT_KEY ni claves privadas de servicios.

Ejecución local

Desde la carpeta principal, se instala y prepara el backend. Antes de migrar se confirma que DATABASE_URL apunte a la base deseada, porque el comando modifica su esquema. migrate.ts lee schema.sql y lo ejecuta; no es un sistema de migraciones versionadas por archivos separados.

```
cd backend
npm ci
npm run migrate
npm run dev
```

En otra terminal, desde la raíz del proyecto, se inicia el cliente:

```
cd frontend
npm ci
npm run dev
```

Para desarrollo, CLIENT_ORIGIN debe coincidir con el origen que muestre Vite, normalmente http://localhost:5173. VITE_API_URL puede ser http://localhost:4000/api. Google debe tener autorizados los orígenes que se usen. El cliente y servidor deben compartir el mismo identificador de Google.

Publicación y actualización

En Render se usa backend como directorio raíz, se instalan dependencias, se compila con npm run build y se inicia con npm start. Las variables de producción se configuran en Environment. schema.sql se aplica con npm run migrate cuando corresponda y después de revisar los cambios y respaldar la información.

En Netlify se utiliza frontend como directorio base, npm run build como compilación y dist como carpeta publicada. VITE_API_URL apunta a https://tutoria-conecta.onrender.com/api y CLIENT_ORIGIN en Render apunta a https://tutoria-conecta.netlify.app. Los cambios en variables VITE_ requieren una nueva compilación del frontend.

Guardamos los cambios en Git, pero aprendimos que también debíamos comparar el commit activo en cada servicio. Netlify ya mostraba 41445be mientras Render todavía ejecutaba 43c7616. Desplegamos manualmente el último commit del backend y comprobamos que Render quedara en 41445be. Después revisamos el ingreso, la agenda y los mensajes.

Los enlaces de la instalación son https://tutoria-conecta.netlify.app/ para la plataforma y https://tutoria-conecta.onrender.com/api para la API. El repositorio es https://github.com/fmoralesa6-lgtm/tutoria-conecta. Los tokens y archivos .env no forman parte del código que se comparte.

Pruebas y validación del funcionamiento

Combinamos pruebas automatizadas de reglas con recorridos manuales en la instalación publicada. Las pruebas unitarias comprueban funciones concretas; las manuales verifican la interacción entre cuentas, servicios y pantallas. Un resultado positivo aplica al escenario realizado y no significa que se hayan cubierto todos los casos posibles.

Pruebas automatizadas

npm test dentro de backend ejecuta tsc y cinco pruebas de rules.test.ts. Se comprobaron cuatro semanas con lunes y miércoles, ocho semanas que cruzan de año, una semana con siete días, ajuste de duración dentro de la disponibilidad y distancia geográfica. Las cinco finalizaron aprobadas durante la revisión del ZIP. El frontend también compiló con npm run build.

tests/concurrency.mjs usa TEST_DATABASE_URL. Crea un esquema temporal y enfrenta dos inserciones superpuestas en conexiones distintas. Espera 23P01 y luego permite un intervalo adyacente. Es una prueba de la restricción PostgreSQL; no recorre por sí sola todos los endpoints. Debe ejecutarse en una base de pruebas con los permisos necesarios.

Pruebas manuales realizadas

Tabla 11

Escenarios comprobados en la plataforma

Escenario	Resultado observado
Nuevo tutor	Perfil pendiente, aprobación, publicación y aparición en catálogo.
Filtros	Precio, modalidad, categoría, radio, reseñas y estrellas según los casos revisados.
Ruta a domicilio	Trayectoria visible y desglose de clases y traslado.
Solicitudes concurrentes	Una solicitud aceptada y otra rechazada por traslape del mismo tutor.
Agenda del estudiante	Mismo horario bloqueado para estudiante ocupado y disponible para otro libre.
Cancelación	Liberación de horarios pendientes o aprobados sin borrar otros bloques.
Chat y archivos	Mensajes, lectura y descarga de PDF desde un participante.
Acceso ajeno	403 en mensajes y adjuntos de otra conversación.
Roles	403 al consultar categorías administrativas como estudiante y tutor.
Sesión vencida	Salida al ingreso tras probar una duración reducida; se restauró a 7200.
Moderación	Reporte, suspensión, apelación aceptada y reactivación.
Reseñas e indicadores	Calificación visible y cambio de tablas y gráficos por mes.
Diseño y despliegue	Recorridos en móvil, temas y navegación tras el último despliegue.

Después de publicar el ajuste de Authorization, se comprobó que el ingreso y la navegación siguieran funcionando. Esa prueba verifica compatibilidad del cliente con el formato requerido. La validación de rechazo de encabezados sin Bearer está respaldada por el middleware; no se documenta aquí como una nueva prueba HTTP ejecutada en producción.

Problemas que encontramos durante las pruebas

El aviso de ingreso seguía visible

Al probar una sesión vencida, la pantalla mostraba el error de sesión y todavía conservaba el aviso de ingreso exitoso. Ajustamos la pantalla de acceso para mostrar el error correspondiente y quitamos el aviso de éxito que ya no describía el estado actual. En api.ts atendimos el 401 antes de procesar el JSON, eliminamos el token y emitimos logout. Aplicamos el mismo manejo a las respuestas de archivos. Al repetir la prueba con una duración corta, comprobamos la salida y restauramos los 7200 segundos.

Dos solicitudes para el mismo horario

Probamos dos cuentas intentando reservar al mismo tutor en el mismo horario, desde tutorías distintas. Una solicitud se guardó y la otra recibió 409 por traslape. No fue una doble reserva que dejamos pasar: fue la comprobación de que la restricción y la transacción funcionaban juntas. También revisamos el caso del estudiante que ya tenía otra clase; a esa cuenta le apareció ocupado el horario y a otra cuenta libre le apareció disponible.

El cambio estaba publicado solo en una parte

La diferencia entre Netlify y Render nos podía hacer probar una pantalla nueva contra un backend anterior. Lo resolvimos comparando los commits activos y desplegando el backend pendiente. Desde esa revisión, comprobar ambos despliegues forma parte de la publicación. Que el sitio abra no basta para afirmar que toda la actualización quedó aplicada.

Lectura de sesiones cortas en el celular

En el calendario móvil, el texto de las sesiones cortas no se leía completo dentro del espacio disponible. Ajustamos los estilos del evento y revisamos su presentación en pantalla

pequeña. Esto nos recordó que compilar sin errores no garantiza que una vista sea cómoda de usar; también tenemos que recorrerla con los datos y el tamaño de pantalla correspondientes.

Mantenimiento y límites de la implementación

Mantenimiento básico

Antes de cambiar schema.sql se debe respaldar la base y revisar cómo afecta los datos existentes. El proyecto concentra creación de tablas y ajustes en ese archivo, por lo que conviene probar cualquier modificación en una instancia separada. Después se compilan frontend y backend y se revisa la función modificada con una cuenta del rol correspondiente.

Es importante conservar por separado las claves privadas y controlar quién accede a las variables de Render. Un cambio de JWT_SECRET invalida los tokens actuales; un cambio de DOCUMENT_KEY requiere tratar los documentos ya cifrados. Las copias de la base deben incluir también chat_attachments porque los archivos se almacenan dentro de PostgreSQL.

Para diagnosticar problemas se revisan los registros de Render y el estado de las solicitudes del navegador. No deben copiarse tokens, identificaciones o cadenas de conexión en reportes públicos. Los errores de rutas o correos se investigan también en sus proveedores, porque no todos dependen del código de la plataforma.

Límites actuales

No existe pago en línea ni renovación automática de suscripciones: los bloques se solicitan y aprueban dentro de la aplicación. El calendario, la reportería y los tableros consultan datos por HTTP; el chat es el componente que utiliza actualizaciones por Socket.io. El ranking conserva el alcance histórico descrito, aunque se cambie el mes de otros indicadores.

La disponibilidad mostrada puede cambiar antes de confirmar; por eso la reserva siempre se valida en el servidor. Las solicitudes a servicios externos tienen tiempo límite. La instancia

gratuita de Render puede demorar al reactivarse tras inactividad, lo cual debe considerarse al preparar la demostración.

Los adjuntos tienen revisión de formato y tamaño, pero no análisis antivirus. La prueba de compilación y los escenarios funcionales no sustituyen una auditoría de seguridad. Como mantenimiento futuro, pueden incorporarse pruebas de integración automatizadas, migraciones versionadas y monitoreo más detallado, sin presentarlos como funciones ya implementadas.

Conclusiones

Con Tutoría Conecta logramos reunir las funciones principales de contratación de tutorías en una aplicación con tres roles. La separación entre pantallas, API y base de datos permite ubicar dónde se presenta la información, dónde se aplican permisos y dónde se conserva cada registro. Esto también facilita explicar el código sin depender únicamente de lo que se ve en pantalla.

La agenda nos dejó una lección concreta: mostrar horarios disponibles y proteger una reserva son tareas distintas. Al registrar a los dos participantes en session_slots y guardar todo en una transacción, protegemos tanto al tutor como al estudiante. La prueba de dos solicitudes para un mismo horario confirmó el rechazo del traslape sin guardar un bloque parcial.

TypeScript nos ayuda a mantener los contratos del código y SCSS permite reutilizar estilos y adaptar la interfaz. Las pruebas con distintas cuentas permitieron comprobar reservas, permisos y moderación. Para continuar el proyecto debemos conservar esa separación de responsabilidades y probar los cambios antes de publicarlos.